- Durante el examen no se responden preguntas y tiene 1 hora y 45 minutos para resolverlo.
- No puede usar aparatos electrónicos y el examen es individual.
- **Por favor disfrutar el examen**
- Recuerde: *"La vida es un carnaval " Ruben Blades*

Ejercicio 1. (2 puntos) Los tres primeros números de la sucesión *TresFibonacci* son $1, 1, 1$ y cada término siguiente es la suma de los tres anteriores. Por ejemplo, los primeros ocho números de la sucesión son:

$$1,\ 1,\ 1,\ 3,\ 5,\ 9,\ 17,\ 31,\ \ldots$$

1. Escriba en python un **generador** llamado *G1()* que genere todos los números de *TresFibonacci*.
2. Escriba en Python un **generador** llamado *tresfibonacci(n)* que produzca (uno por uno) los primeros n números de la sucesión TresFibonacci.

In [ ]

```
Ge, Ge1 = G1(), tresfibonacci(5)
for i in range(6):
    print(next(Ge), end = ' ')
> 1 1 1 3 5 9

for i in Ge1:
    print(i, end = ' ')
> 1 1 1 3 5
```

Ejercicio 2. (1.5 puntos) Crear un decorador *f2(func)* que permite al usuario utilizar las función y cometer tres errores de ejecución (por ejemplo *ValueError*). Si no existen errores de ejecución retorna el resultado. Sin embargo, al cometer un error se debe imprimir el error y un mensaje con el número de errores cometido hasta ese momento y no debe retornar nada. A partir del tercer error no permite seguir usando la función y después de eso debe mostrar siempre el mensaje *Leer la documentación*.

In [ ]

```
@f2
def division(a,b):
    return a/b

division(1,2), division(1,0), division('a', 1), division('a', 'b'), division(1,1)
> Ha cometido 1 error(es): ZeroDivisionError
> Ha cometido 2 error(es): TypeError
> Ha cometido 3 error(es): TypeError
> Leer la documentacion
(0.5, None, None, None, None)
```

Ejercicio 3. (1.5 puntos) Escribir un función iterativa *f3i(n)* y una función recursiva *f3r(n)* que recibe un número natural diferente de 0 y retorna el último dígito del número. Nota: No se puede usar *str* :(

In [ ]

```
L = [4, 23, 189]
for i in L:
    print(f3i(i), end = ' ')
> 4 2 1
```